Lab 3 Report: Wall Follower

Team 2

Brendon Murphy
Nina Ruiz-Garcia
Irene Tang
Vandell Vatel

RSS

March 9, 2026

1 Introduction

Author: Vandell Vatel

A safety controller is an important feature of autonomous vehicles that helps them prioritize the safety of passengers and bystanders through a program that tells them to stop if their continued movement would cause them to crash. Through this lab, we were tasked with the development of a safety controller program that would supersede our previously written wall follower program when necessary to prevent our race-car from crashing into obstacles. The technicality lies in tuning our safety controller so that it strikes a balance of risk-aversion and risk-tolerance. If the controller is too risk-averse, it will cause our race-car to stop when it is not in danger of crashing, causing it to perform sub-optimally in future labs. However, if it is too risk-tolerant, it either won't stop or won't stop early enough to prevent a crash. Our solution is a safety controller that constantly calculates a projected path that our race-car would take in a short time frame. If an obstacle is detected within a designated safety radius, meaning that we would collide with the object within the time frame, the controller takes priority and forces the race-car to stop.

2 Technical Approach

This section summarizes the control architecture used for two key vehicle functions: wall following and safety control.

Wall Following

Author: Irene Tang

The goal of the wall follower is to maintain a set distance away from a wall while driving beside it. The input signal for the car is the necessary steering angle to maintain the appropriate distance. To achieve this, the wall-following pipeline does two things:

- Wall Estimator: fits a line to lidar measurements corresponding to the nearby wall.
- Path Controller: computes steering commands to maintain a desired offset from the wall while aligning the vehicle with it.

1. Wall Estimator

The estimator constructs a linear approximation of the wall using a subset of lidar scan points. Only scan points likely to belong to the side wall are retained. We restrict measurements to an angular window $[-\pi/12, \pi/2]$ if following the left wall or $[-\pi/12, \pi/2]$ if following the right wall and distance range $[0.2, 5]$ m to remove irrelevant returns. The angular range includes angles in front of the car to estimate corners and handle turning behavior. A straight line is then fit to the filtered points $\{(x_i, y_i)\}$ using ordinary least squares:

$$\begin{gathered}\theta_{left} \in [-\pi/12, \pi/2] \\ \theta_{right} \in [-\pi/2, \pi/12] \\ d \in [0.2\text{m}, 5\text{m}] \\ (m, b) = \arg\min_{m,b} \sum_i (y_i - mx_i - b)^2 \end{gathered} \tag{1}$$

This line provides two quantities used by the controller:

- The perpendicular distance from the vehicle to the wall

$$d = \frac{-b}{\sqrt{m^2 + 1}} \tag{2}$$

- The wall orientation relative to the vehicle

$$\theta_{\text{wall}} = \arctan(m) \tag{3}$$

These values form the wall state estimate used for feedback control.

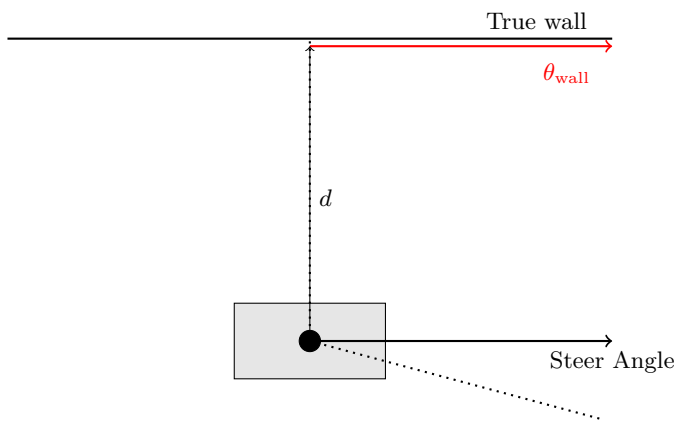

Figure 1: **Wall Estimator Visualization.** The dotted lines show the angular range of lidar points considered for wall estimation. The red arrow represents our estimated wall relative to the true wall. d is the perpendicular distance from the car to the estimated wall, and θ_{wall} is the wall orientation relative to the car frame.

2. PD Controller

Steering commands are generated using a proportional-derivative (PD) controller that regulates the lateral distance between the vehicle and the estimated wall. Let $s \in \{-1, +1\}$ denote the tracking side ($+1$ for left, -1 for right), d^* the desired wall distance, and d the estimated distance. The signed distance error is defined as

$$e = -sd^* - d \tag{4}$$

The derivative term is computed using the controller update period Δt:

$$\dot{e} = \frac{e - e_{\text{prev}}}{\Delta t}, \qquad \Delta t = 0.02 \text{ s} \tag{5}$$

The final steering command combines distance regulation and wall alignment:

$$\delta = K_p e + K_d \dot{e} + K_w \theta_{\text{wall}} \tag{6}$$

where K_p, K_d, and K_w are controller gains. From tuning in simulation, we discovered the gains to be around 2, 0.2, and 2 respectively.
To respect the vehicle's physical limits, the command is saturated to the maximum steering angle $\delta_{\max}$:

$$\delta_{\text{cmd}} = \text{clip}(\delta, -\delta_{\max}, \delta_{\max}) \tag{7}$$

This control structure maintains a fixed offset from the wall while encouraging the vehicle to remain parallel to it.

Safety Controller

Author: Vandell Vatel

Our safety controller works by "catching" driving commands that are being published, and stopping the race-car if a command would cause it to crash within a given time-frame. It works by projecting a path that our race-car would follow, and ensuring that any obstacles detected by our LiDAR are far enough away from the path to not be hit, creating a "safety radius" around our car.

1. Path Projection

The path projection computes a line that the race-car will follow based on the velocity extracted from the intercepted drive command. For now, we have simplified the projected path to be forward, in the x-direction, from the frame of the race-car. Since we run the line projection every time a drive command is published, at a rate of 50 Hz, we assume slight deviations in the race-car's angle with respect to the line will be caught by future iterations. The computed line is therefore a straight line forward from the race-car that spans a projected distance d_{proj} as a function of Δt, the time frame of the projection, v, the velocity of the race-car, and r_{safe}, the safety radius of our car:

$$d_{proj} = \Delta t * v + r_{safe} \tag{8}$$

$$\vec{v}_{path} = [d_{proj}, 0, 0] \tag{9}$$

The values of Δt and r_{safe} were tuned through testing detailed later in Experimental Evaluation.

2. Crash Detection

Taking this projected path, we check if there are any obstacles that our race-car may crash into it, and if so, issue a command overriding the intercepted drive command, telling our race-car to stop. The most recent LiDAR data is filtered to only include points that are within angle ranges [-$pi/4$, $pi/4$] and that are a distance d_{proj} away from the race-car shown in Figure 2 as a cone in front of the car. We assume that points filtered out by this are too far from the path to ever realistically be collided with. For each of the filtered points, we calculate the cross product of the point p and the normalized path vector to find the distance of the point from the projected path.

$$\delta_p = p \times \frac{\vec{v}_{path}}{|\vec{v}_{path}|} \tag{10}$$

$$\exists p : \delta_p < r_{safe} \tag{11}$$

If the δ_p of any of these points is less than r_{safe}, it would mean that there are obstacles in the current projected path of the race-car, that are not far away enough to not be crashed into. This is shown in the left scenario of Figure 2, where the red circle represents r_{safe} and the red arrow represents a $\delta_p < r_{safe}$.

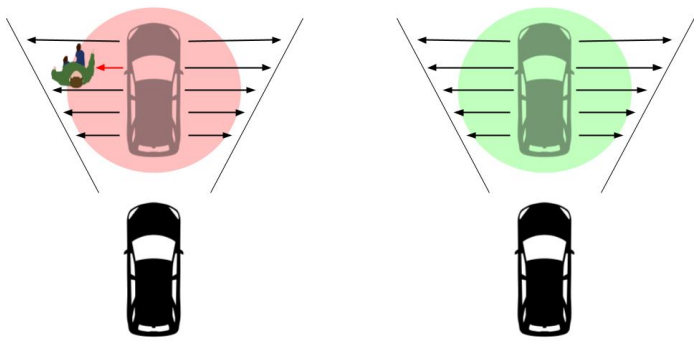

Figure 2: **Crash Detection Visualization.**
(*Left*) A bystander is detected within the safety radius of the race-car.
(*Right*) No obstacles are found and the race-car is free to proceed.

3 Experimental Evaluation

Just detailing our tests with figures and graphs
Author: Team Member
(up to 700 words)

3.1 Wall Follower Evaluation

3.2 Safety Controller Evaluation

Safety error is defined as

$$e_{safety} = |d_{desired} - d_{observed}|$$

from when the rover has reached a complete stop.

4 Conclusion

Author: Team Member
(up to 275 words)

5 Lessons Learned

Presents individually authored self-reflections on technical, communication, and collaboration lessons you have learned in the course of this lab.

6 Formatting examples for figures, pseudocode, etc

6.1 Tables

Many other table packages and options exist but here is one example:

item 11	item 12	item 13
item 21	item 22	item 23

6.2 Images

Figure 3: Figure caption.

6.3 Code Blocks and Algorithm Pseudocode

```
json
{
  "6.141": "normal",
  "16.405": "woke",
  "no_sleep": "spoke"
}

def do_something_productive():
  if not_productive:
    do_work()
  else:
    cry()
```

```
while alive do
    if sleepy then
        sleep;
    else
        eat;
    end
end
```

Algorithm 1: caption

	$x = 2$	$x = 1.5$	$x = 1$
$v = 1$	good	good	good
$v = 2$	too close!	too close!	okay
$v = 3$	too far!	too far!	okay

```
i ← 10
if i ≥ 5 then
  i ← i − 1
else
  if i ≤ 3 then
    i ← i + 2
  end if
end if
```